IF2224 TEORI BAHASA FORMAL DAN OTOMATA

TUGAS BESAR
Milestone 1

(Kelompok TBFO)

Dipersiapkan oleh:

13524044	Narendra Dharma Wistara M
13524060	Steven Tan
13524090	Nashiruddin Akram
13524100	Arghawisesa Dwinanda Arham

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

Daftar Isi **1**
Teori Singkat **2**
Analisis Leksikal 2
Token 2
Deterministic Finite Automata (DFA) 3
Perancangan dan Implementasi **4**
Diagram Transisi DFA 5
Implementasi 10
Pengujian **15**
Pengujian 1 15
Pengujian 2 15
Pengujian 3 16
Pengujian 4 17
Pengujian 5 18
Kesimpulan dan Saran **19**
Kesimpulan 19
Saran 19
Lampiran **20**
Referensi **21**

Teori Singkat

Bab ini menyajikan teori singkat mengenai proses analisis leksikal sebagai tahap pertama dalam kompilasi, token, serta prinsip kerja Deterministic Finite Automata (DFA) yang digunakan untuk memodelkan pengenalan token secara formal.

Analisis Leksikal

Analisis leksikal adalah tahap awal proses kompilasi yang bertugas mengubah rangkaian-rangkaian karakter dari source program menjadi rangkaian-rangkaian token. Pada tahap ini, masukan dibaca dari kiri ke kanan, lalu dikelompokkan ke tipe token yang sesuai.

Dalam kompiler/interpreter, lexer bertindak sebagai jembatan antara source code dengan parser. Lexer memproses karakter-karakter dalam source code menjadi token yang mudah dimengerti parser. Dengan adanya lexer, parser cukup menerima token sehingga analisis sintaksis menjadi lebih efisien.

Lexer memiliki empat fungsi utama:

- Membagi bagian dari input menjadi token
- Mengklasifikasikan token ke tipe yang tepat
- Melakukan penyaringan dengan mengabaikan karakter yang tidak bermakna seperti spasi, tab, dsb.
- Menyajikan hasil tokenisasi sebagai masukan bagi parser pada tahap analisis sintaksis.

Token

Token adalah unit terkecil yang bermakna dalam proses kompilasi. Token bisa dipandang sebagai pasangan antara pola karakter dan representasi leksikalnya.

Secara umum, token terdiri dari:

$$Token = (type, lexeme, attribute)$$

dengan:

- *type*: kategori token, misalnya identifier, keyword, komentar, operator, literal, atau tanda baca.
- *lexeme*: substring pada source code yang cocok dengan pola karakter tipe token tersebut.
- *attribute*: informasi tambahan, seperti posisi baris untuk kebutuhan pelaporan error dan tahap lanjutan.

Deterministic Finite Automata (DFA)

Secara formal, sebuah DFA didefinisikan sebagai 5-tuple:

$$M = (Q, \Sigma, \delta, q, F)$$

dengan:

- Q: himpunan state.
- Σ: alfabet input (kumpulan karakter yang diperbolehkan).
- δ: fungsi transisi yang mengambil state dan simbol input, kemudian mengembalikan sebuah state
- $q0$: state awal.
- F: himpunan state final.

Deterministik di sini berarti untuk setiap pasangan (q, a) dengan $q\ in\ Q$ dan $a\ in\ \Sigma$, terdapat tepat satu state tujuan $\delta(q, a)$. Tidak ada transisi ke berbagai state yang berbeda untuk input yang sama pada state yang sama.

Dalam konteks lexer, setiap tipe token dapat dimodelkan sebagai DFA (atau bagian dari DFA gabungan). Alurnya:

- Lexer mulai dari $q0$.
- Membaca karakter satu per satu.
- Berpindah state sesuai fungsi transisi δ.
- Berhenti ketika tidak ada transisi valid.
- Menerima lexeme jika state terakhir berada di F.

Bahasa token yang dikenali oleh DFA adalah:

$$L(M) = \{ w \mid \hat{\delta}(q0, w)\ in\ F \}$$

Kaitan DFA dengan token dan lexer dapat dipahami sebagai berikut:

- DFA menjadi model untuk mendefinisikan aturan/pola yang berlaku pada token.
- Lexer adalah pelaksana yang mengeksekusi aturan transisi tersebut pada input.
- Token adalah hasil akhir ketika suatu string input berhasil diterima automata.

Perancangan dan Implementasi

Bagian ini mendeskripsikan perancangan diagram transisi DFA untuk berbagai kategori token dalam bahasa Arion, mulai dari konstanta bilangan hingga keyword kontrol alur, serta penjelasan mengenai implementasi kode yang digunakan dalam membangun lexer.

Pada perancangan DFA, state START dibuat banyak untuk menyederhanakan diagram dan memudahkan pembagian tugas. Secara konsep, sebenarnya cukup menggunakan satu state START untuk seluruh token. Namun, jika semua token digabung dalam satu diagram dengan satu state awal, maka diagram akan menjadi sangat besar, kompleks, dan sulit dibaca.

Dari state START, DFA dapat menerima berbagai kemungkinan karakter (seperti huruf, digit, maupun simbol), lalu berpindah ke jalur yang sesuai hingga mencapai state *accept* yang merepresentasikan suatu token. Setelah sebuah token dikenali, proses pembacaan dilanjutkan kembali dari state START untuk token berikutnya.

Meskipun pada diagram transisi tidak selalu ditampilkan panah eksplisit kembali ke START, perilaku tersebut tetap ada dalam implementasi melalui proses pembacaan berulang terhadap input. Dengan demikian, desain ini tetap merepresentasikan DFA yang memproses seluruh *source code* secara berurutan, sekaligus menjaga kesederhanaan dan keterbacaan diagram.

Diagram Transisi DFA

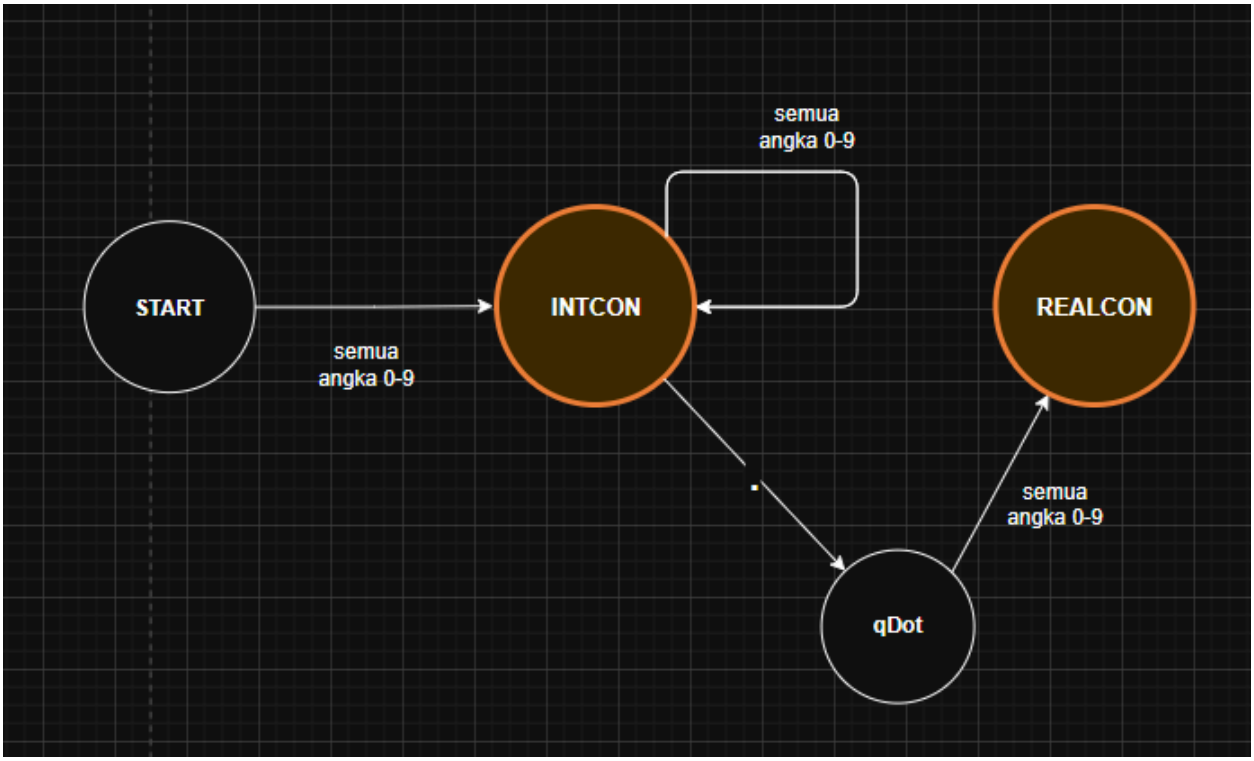

Diagram menunjukkan DFA untuk mengenali bilangan, di mana dari state awal jika membaca digit akan masuk ke **INTCON** dan tetap di sana selama masih digit sehingga menghasilkan bilangan bulat. Jika ditemukan tanda titik (.), DFA berpindah dan jika diikuti digit akan masuk ke **REALCON** untuk menghasilkan bilangan real.'

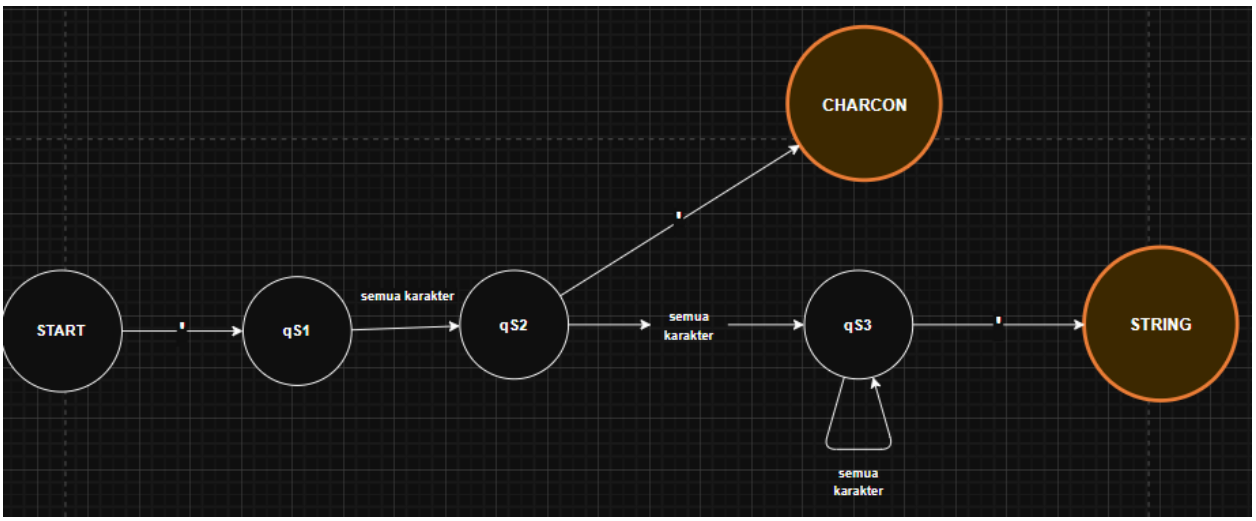

Diagram menunjukkan DFA untuk **char** dan **string** yang diawali dan diakhiri tanda petik ('). Setelah membaca petik awal, DFA membaca isi karakter, jika hanya satu karakter maka

menjadi **CHARCON**, sedangkan jika lebih dari satu karakter maka menjadi **STRING**, dengan pembacaan berulang hingga menemukan petik penutup.

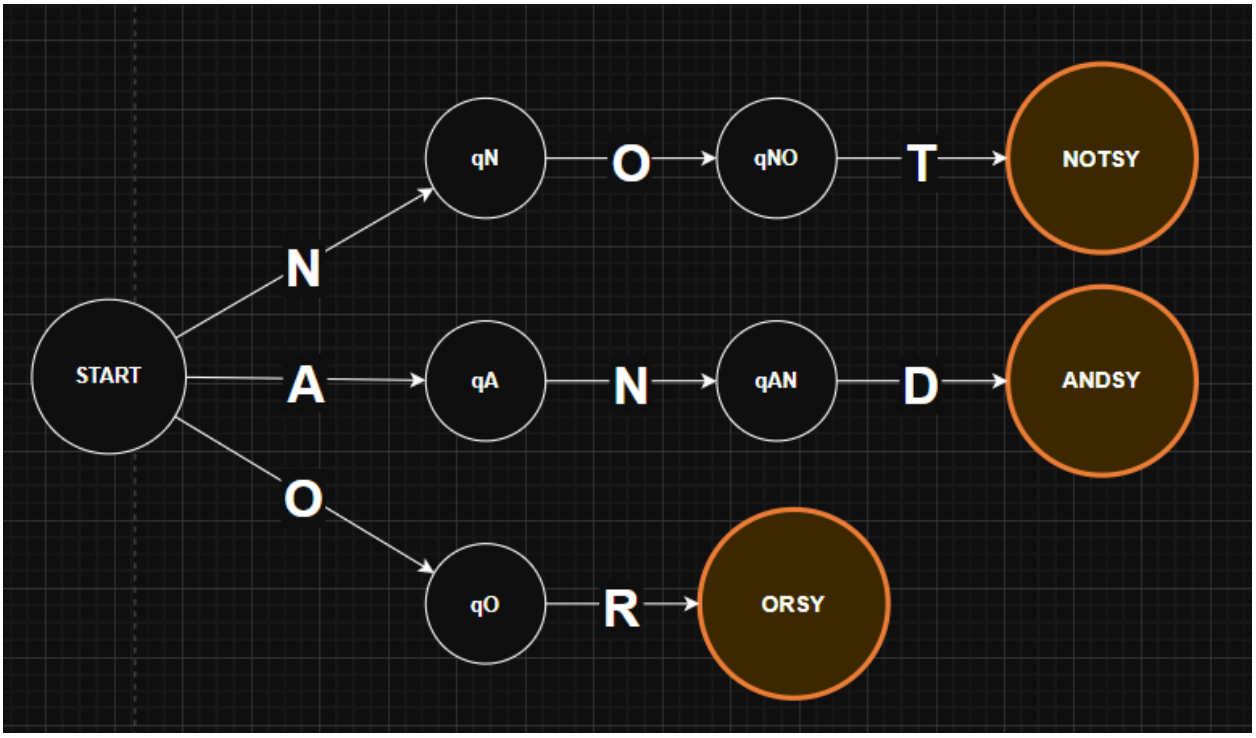

Diagram menunjukkan DFA untuk operator perbandingan seperti =, <, dan >, di mana DFA mengecek apakah operator terdiri dari satu atau dua karakter. Misalnya = yang diikuti = menjadi EQL (==), < bisa menjadi LSS (<), LEQ (<=), atau NEQ (< >), dan > bisa menjadi GTR (>) atau GEQ (>=).

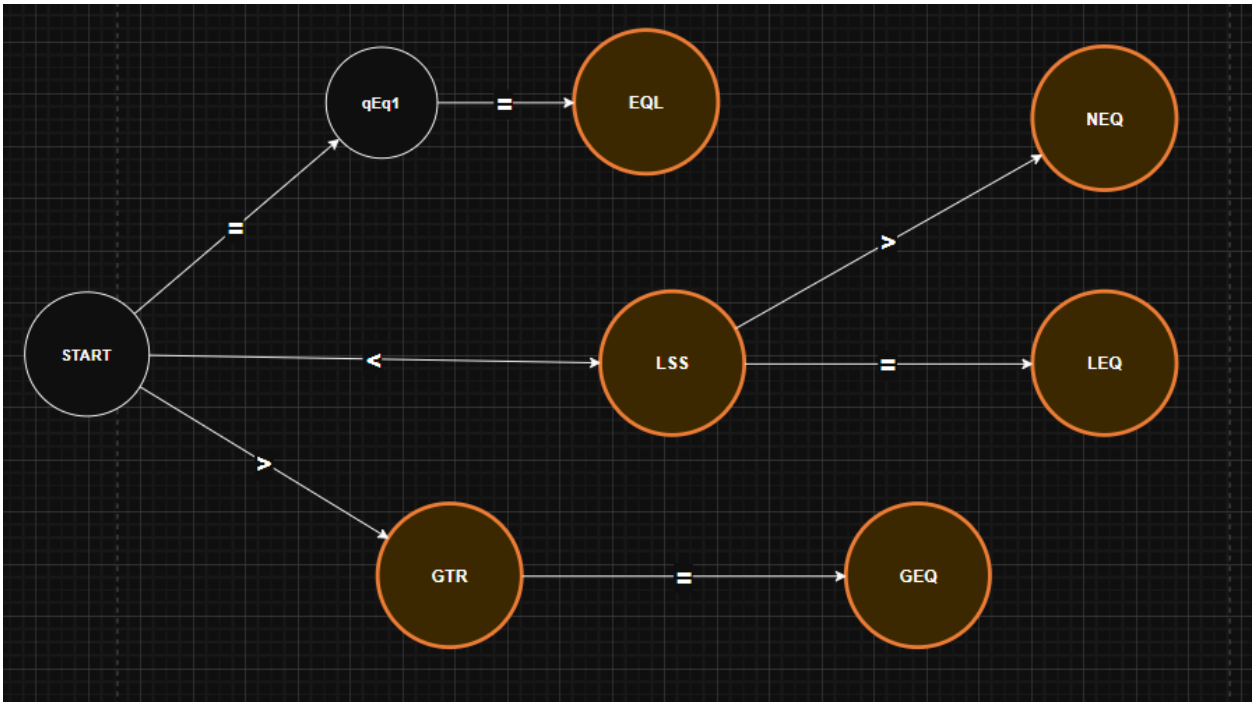

Diagram menunjukkan DFA untuk keyword logika seperti **NOT**, **AND**, dan **OR**, yang dibaca huruf per huruf dari state awal. Jika urutan huruf sesuai, maka akan menghasilkan token seperti **NOTSY**, **ANDSY**, atau **ORSY**, sehingga DFA berfungsi untuk mengenali keyword secara spesifik.

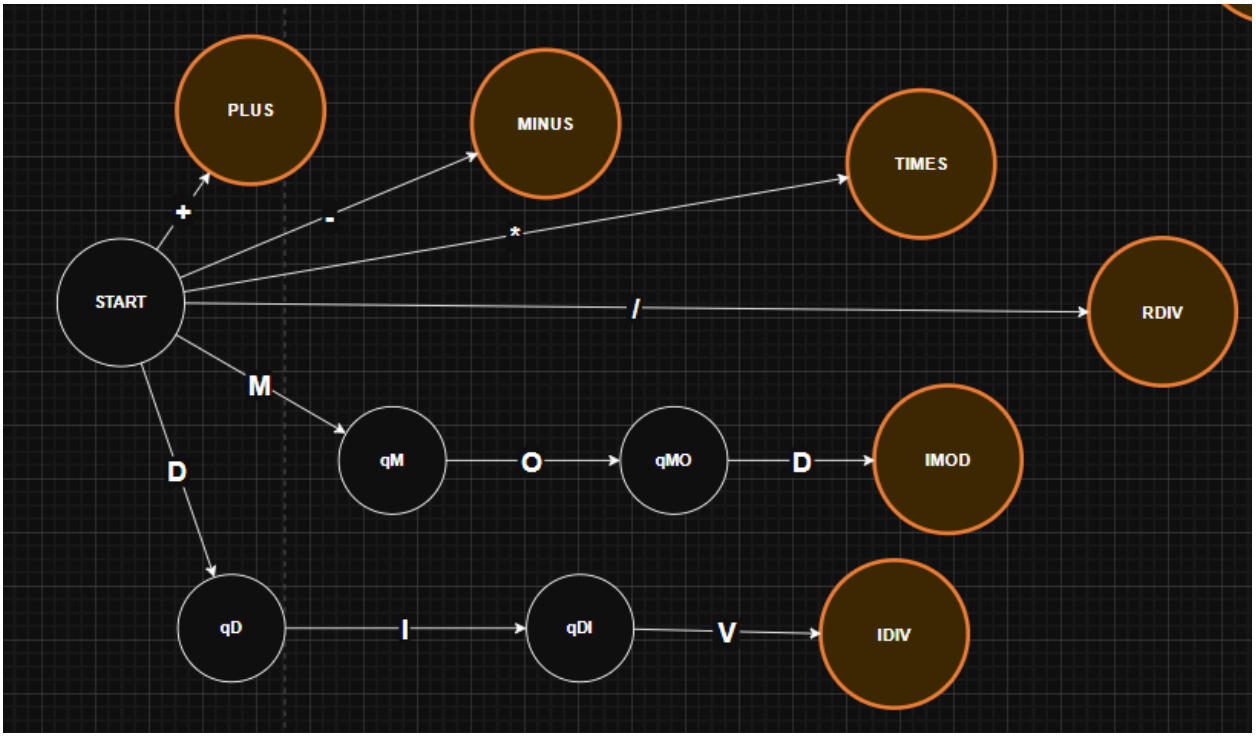

Diagram menunjukkan DFA untuk operator perbandingan seperti =, <, dan >, di mana DFA mengecek apakah operator terdiri dari satu atau dua karakter. Misalnya = yang diikuti = menjadi **EQL (==)**, < bisa menjadi **LSS (<)**, **LEQ (<=)**, atau **NEQ (<>)**, dan > bisa menjadi **GTR** ⊙ atau **GEQ (>=)**.

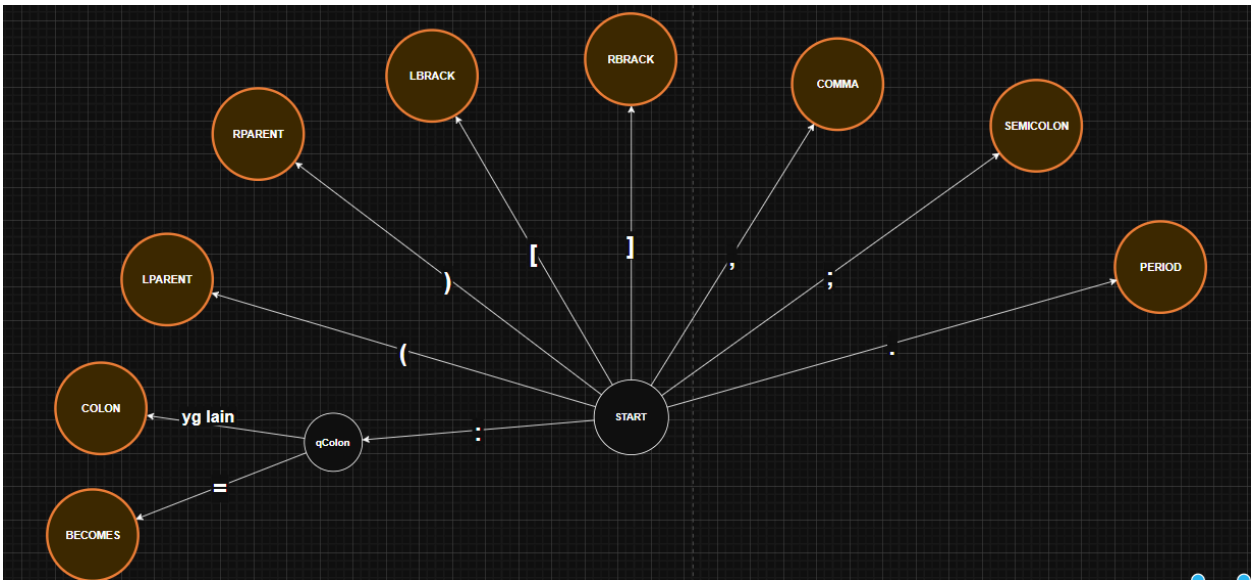

Diagram menunjukkan DFA untuk berbagai tanda baca dan operator sederhana yang langsung dikenali dari satu karakter. Dari state awal, simbol seperti (,), [,], ,, ;, dan . masing-masing langsung berpindah ke state akhir seperti **LPARENT**, **RPARENT**, **LBRACK**, **RBRACK**, **COMMA**, **SEMICOLON**, dan **PERIOD** tanpa perlu membaca karakter tambahan. Untuk simbol titik dua (:), DFA memiliki pengecekan lanjutan. Jika hanya : maka menghasilkan **COLON**, tetapi jika diikuti = maka menjadi **BECOMES (:=)**.

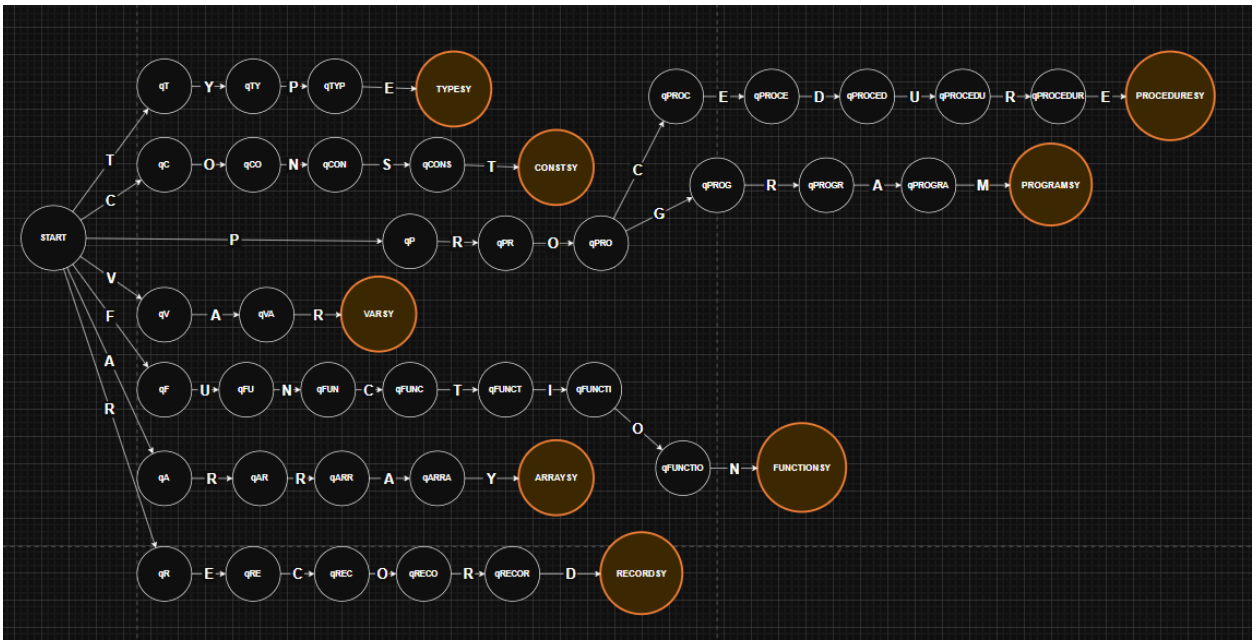

Diagram menunjukkan DFA untuk keyword deklarasi seperti **TYPE**, **CONST**, **VAR**, **ARRAY**, **RECORD**, **PROGRAM**, **PROCEDURE**, dan **FUNCTION**, yang dikenali dengan membaca huruf satu per satu dari state awal hingga membentuk kata lengkap. Setiap urutan huruf yang sesuai akan berakhir di state final dan menghasilkan token keyword terkait, sedangkan jika tidak cocok maka tidak dianggap sebagai keyword.

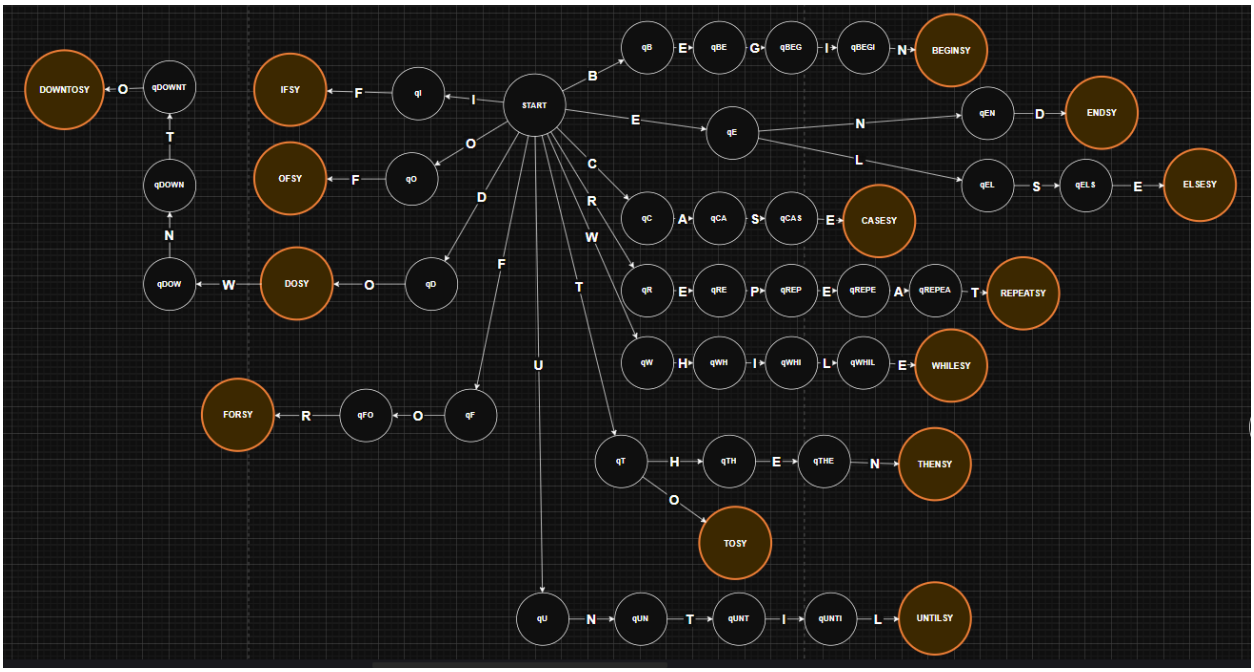

Diagram menunjukkan DFA untuk keyword kontrol alur seperti **BEGIN**, **END**, **IF**, **ELSE**, **CASE**, **REPEAT**, **WHILE**, **FOR**, **DO**, **TO**, **DOWNTO**, dan **UNTIL**. Prosesnya serupa, yaitu

membaca karakter secara berurutan hingga membentuk kata tertentu, lalu menghasilkan token sesuai keyword tersebut jika seluruh pola terpenuhi.

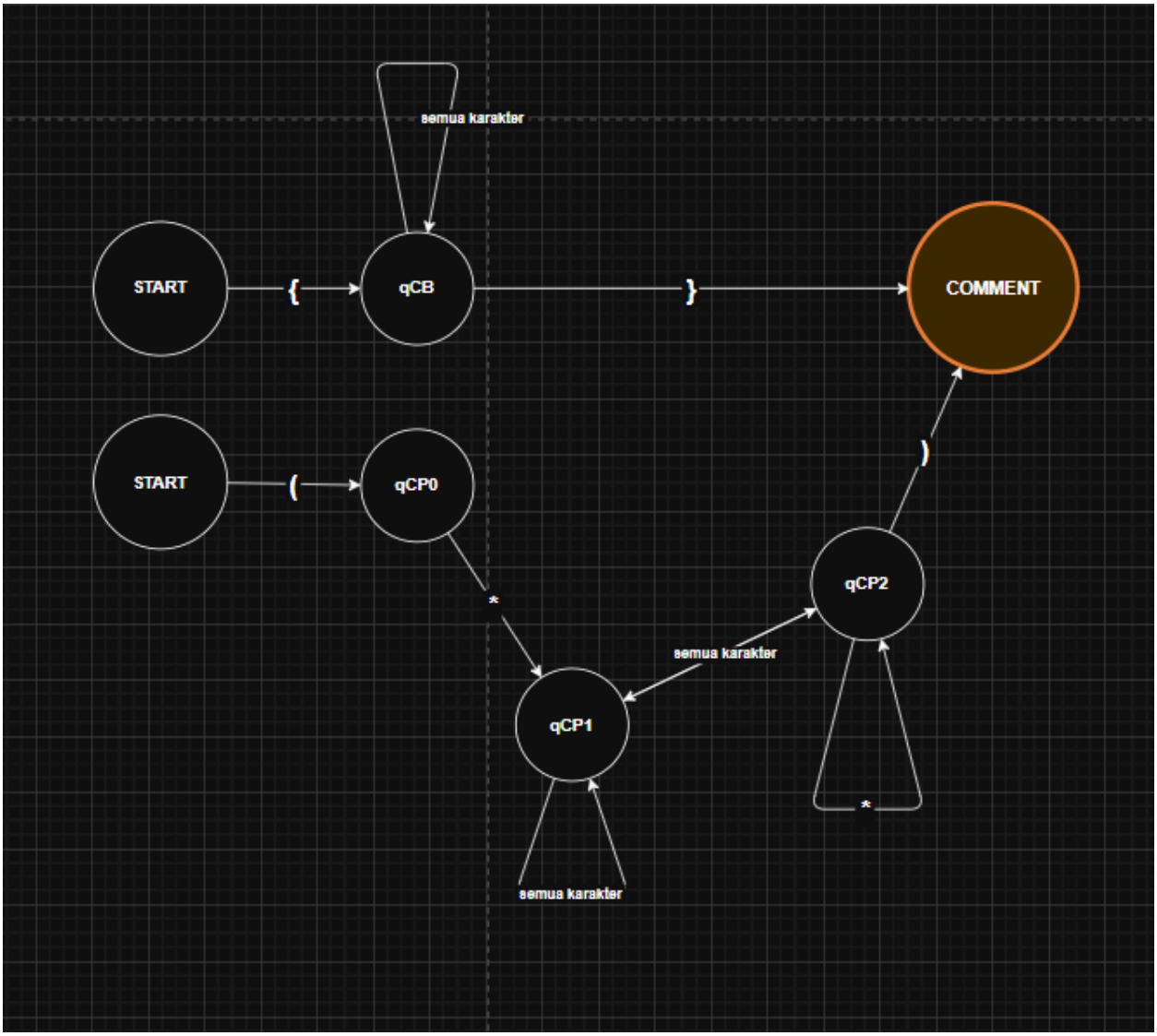

Diagram menunjukkan DFA untuk **comment** yang diawali { ... } atau (* ... *). DFA membaca semua karakter hingga menemukan penutup yang sesuai (} atau *)), lalu menghasilkan token **COMMENT**.

Implementasi

```
PSEUDOCODE

TOKENIZE():
  // DFA state utama: S0 (start)
  tokens <- []
  nextChar() // ambil karakter pertama
```

```
  while not EOF:
    c <- currentChar

    if c adalah whitespace:
      nextChar()// tetap di S0
      continue

    else if c adalah huruf:
      tokens.append(READ_IDENT_OR_KEYWORD()) // S_IDENT
      continue

    else if c adalah digit:
      tokens.append(READ_NUMBER()) // S_INT / S_REAL
      continue

    else if c == '\'':
      tokens.append(READ_STRING_OR_CHAR()) // S_STR
      continue

    else:
      tokens.append(READ_OPERATOR_OR_PUNCT()) // S_OP

  return tokens


LOOKUP_KEYWORD(wordLower):
  if wordLower sesuai dengan salah satu keyword: (if, while, begin,
end, not, and, or, dst):
    return TOKEN_KEYWORD_NYA
  else:
    return IDENT


READ_IDENT_OR_KEYWORD():
  word <- ""

  while currentChar (huruf atau angka) atau '_':
    word <- word + currentChar
    nextChar()

  type <- LOOKUP_KEYWORD(lowercase(word))
  if type == IDENT:
    return Token(IDENT, word)
  else:
    return Token(type)


READ_NUMBER():
```

```
    num <- ""

    while currentChar digit:
      num <- num + currentChar
      nextChar()

    if currentChar == '.':
      num <- num + '.'
      nextChar()

      if currentChar bukan digit:
        return Token(TOKEN_ERROR)

      while currentChar digit:
        num <- num + currentChar
        nextChar()

      return Token(REALCON, num)

    return Token(INTCON, num)


  READ_STRING_OR_CHAR():
    nextChar() // lewati quote awal
    content <- ""

    while true:
      if EOF:
        return Token(TOKEN_ERROR)

      if currentChar == '\'':
        nextChar()
        if currentChar == '\'': // escape ''
          content <- content + '\''
          nextChar()
        else:
          break // quote penutup
      else:
        content <- content + currentChar
        nextChar()

    if length(content) == 1:
      return Token(CHARCON, content)
    else:
      return Token(STRING, content)


  READ_OPERATOR_OR_PUNCT():
    switch currentChar:
```

```
    case '+','-','*','/','(',')','[',']',',',';','.':
      t <- simbolKeToken(currentChar)
      nextChar()
      return Token(t)

    case ':':
      if peek() == '=':
        nextChar()
        nextChar()
        return Token(BECOMES) // :=
      nextChar()
      return Token(COLON)

    case '>':
      nextChar()
      if currentChar == '=':
        nextChar()
        return Token(GEQ) // >=
      return Token(GTR) // >

    case '<':
      nextChar()
      if currentChar == '=':
        nextChar()
        return Token(LEQ) // <=
      if currentChar == '>':
        nextChar()
        return Token(NEQ) // <>
      return Token(LSS) // <

    case '=':
      nextChar()
      if currentChar == '=':
        nextChar()
        return Token(EQL) // ==
      return Token(TOKEN_ERROR)

    default:
      nextChar()
      return Token(TOKEN_ERROR)
```

Implementasi program dilakukan dengan membuat lexical analyzer berbasis DFA secara handwritten, di mana setiap state tidak direpresentasikan sebagai tabel transisi, tetapi sebagai kombinasi if/switch dan loop dalam kode. Program utama membuka file input, memanggil lexer untuk melakukan tokenisasi, lalu menampilkan hasil token, dengan

token seperti COMMENT dibuang sebelum output. Struktur token sendiri menyimpan tipe, nilai (jika ada), dan nomor baris.

Pada prosesnya, terdapat satu start state (dispatcher) dalam loop utama tokenisasi yang membaca karakter satu per satu. Jika menemukan whitespace, lexer tetap di state awal (di-skip). Jika menemukan huruf, lexer berpindah ke state identifier/keyword; jika digit ke state number; jika tanda kutip ke state string/char; dan selain itu ke state operator atau tanda baca. Setiap transisi ini bersifat deterministik karena untuk setiap pasangan (state, karakter) hanya ada satu aksi yang mungkin.

Untuk identifier dan keyword, lexer membaca rangkaian huruf/angka, lalu mencocokkannya dengan daftar keyword menggunakan fungsi seperti lookupKeyword(). Secara konsep DFA, ini merepresentasikan banyak state per huruf, tetapi diimplementasikan lebih efisien sebagai satu state S_IDENT dengan proses akumulasi karakter. Untuk angka, lexer membaca digit dan mengecek kemungkinan bilangan real jika terdapat titik. Untuk string dan char, lexer membaca karakter di antara tanda kutip dan menentukan jenis token berdasarkan panjang isi. Operator dan tanda baca ditangani dengan pengecekan satu atau dua karakter, seperti :=, >=, atau <>.

Dengan demikian, implementasi ini tetap mengikuti konsep DFA (deterministik dan berbasis state), tetapi dioptimalkan dalam bentuk kode prosedural agar lebih sederhana dan efisien dibandingkan representasi DFA formal karakter per karakter.

Pengujian

Untuk memastikan bahwa lexical analyzer dapat mengenali seluruh komponen bahasa Arion dengan benar, dilakukan serangkaian pengujian menggunakan berbagai skenario input yang menghasilkan output.

```
arghawisesaarham@Arghawisesas-MacBook-Pro-5 ISL-Tubes-IF2224-2026 % ./lexer test/milestone-1/test-1.txt test/milestone-1/output-1.txt
[OK] Tokenisasi selesai. 2 token ditulis ke 'test/milestone-1/output-1.txt'
arghawisesaarham@Arghawisesas-MacBook-Pro-5 ISL-Tubes-IF2224-2026 % ./lexer test/milestone-1/test-2.txt test/milestone-1/output-2.txt
[OK] Tokenisasi selesai. 30 token ditulis ke 'test/milestone-1/output-2.txt'
arghawisesaarham@Arghawisesas-MacBook-Pro-5 ISL-Tubes-IF2224-2026 % ./lexer test/milestone-1/test-3.txt test/milestone-1/output-3.txt
[OK] Tokenisasi selesai. 19 token ditulis ke 'test/milestone-1/output-3.txt'
arghawisesaarham@Arghawisesas-MacBook-Pro-5 ISL-Tubes-IF2224-2026 % ./lexer test/milestone-1/test-4.txt test/milestone-1/output-4.txt
[OK] Tokenisasi selesai. 14 token ditulis ke 'test/milestone-1/output-4.txt'
arghawisesaarham@Arghawisesas-MacBook-Pro-5 ISL-Tubes-IF2224-2026 % ./lexer test/milestone-1/test-5.txt test/milestone-1/output-5.txt

[LEXER] Ditemukan karakter/token tidak dikenal:
  Baris 2: karakter 'Komentar '{' tidak ditutup'
[LEXER] Proses tokenisasi tetap dilanjutkan.

[OK] Tokenisasi selesai. 2 token ditulis ke 'test/milestone-1/output-5.txt'
arghawisesaarham@Arghawisesas-MacBook-Pro-5 ISL-Tubes-IF2224-2026 %
```

Pengujian 1

Diberikan input sebagai berikut:

```
{ test }
(* test *)
```

Dihasilkan output sebagai berikut:

```
comment({ test })
comment((* test *))
```

Pengujian 2

Diberikan input sebagai berikut:

```
program Hello;
var
a, b: integer;
begin
a := 5;
b := a + 10;
writeln('Result = ', b);
end.
```

Dihasilkan output sebagai berikut:

```
programsy
ident (Hello)
semicolon

varsy
```

```
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon

beginsy
ident (a)
becomes
intcon (5)
semicolon

ident (b)
becomes
ident (a)
plus
intcon (10)
semicolon

ident (writeln)
lparent
string ('Result = ')
comma
ident (b)
rparent
semicolon

endsy
period
```

Pengujian 3

Diberikan input sebagai berikut:

```
IF x > 0 THEN
begin
  while x > 0 do
    x := x - 1;
end;
```

Dihasilkan output sebagai berikut:

```
ifsy
ident (x)
gtr
intcon (0)
thensy
```

```
beginsy
whilesy
ident (x)
gtr
intcon (0)
dosy
ident (x)
becomes
ident (x)
minus
intcon (1)
semicolon

endsy
semicolon
```

Pengujian 4

Diberikan input sebagai berikut:

```
{ ini komentar satu baris }
var x : integer;
(* ini komentar
   multi baris *)
begin
  x := 5;
end.
```

Dihasilkan output sebagai berikut:

```
comment ({ ini komentar satu baris })
varsy
ident (x)
colon
ident (integer)
semicolon

comment ((* ini komentar
   multi baris *))
beginsy
ident (x)
becomes
intcon (5)
semicolon

endsy
period
```

Pengujian 5

Diberikan input sebagai berikut:

```
begin
  { komentar tidak ditutup
  x := 5;
end.
```

Dihasilkan output sebagai berikut:

```
beginsy
unknown (Komentar '{' tidak ditutup)
```

Kesimpulan dan Saran

Berdasarkan hasil pengembangan dan pengujian yang telah dilakukan terhadap lexical analyzer berbasis DFA ini, bagian ini merangkum pencapaian fungsionalitas program serta memberikan rekomendasi untuk optimasi pada milestone pengembangan selanjutnya.

Kesimpulan

Tahap analisis leksikal merupakan tahap awal dan jembatan esensial antara source code dengan parser dalam proses kompilasi. Berdasarkan perancangan dan implementasi yang telah dilakukan, pembangunan lexer untuk bahasa Arion berhasil dimodelkan secara efektif menggunakan Deterministic Finite Automata (DFA). Pemodelan lexer ini lalu diimplementasikan menggunakan bahasa C++ untuk pembentukan kelas yang mendukung modularitas program. Setelah proses implementasi dan pengujian, tidak ditemukan *bug* apapun di dalam program dan program berhasil berjalan dengan lancar.

Saran

Terlepas dari program yang berjalan dengan lancar, terdapat beberapa saran yang dibagi menjadi tiga koridor: kepada kelompok ISL, pembuat spesifikasi Tugas Besar, dan pengembang selanjutnya. Saran-saran tersebut adalah sebagai berikut.

1. Kelompok ISL
   Mekanisme pelaporan kesalahan perlu dioptimalkan agar tidak menyebabkan *crash* dan memberikan pesan yang lebih informatif. Penambahan informasi presisi seperti nomor baris dan kolom saat menemukan masukan yang tidak valid atau *unknown* akan sangat membantu perbaikan *source code*.

2. Pembuat Spesifikasi
   Pendefinisian spesifikasi dapat diperjelas dengan menambahkan contoh penanganan *edge case* pada tabel daftar token, guna menghindari ambiguitas interpretasi antar kelompok yang mengerjakan Tugas Besar.

3. Pengembang Selanjutnya
   Sangat disarankan agar pengembangan lanjutan tetap mempertahankan struktur modular. Hal ini esensial agar daftar token yang telah dihasilkan *lexer* dapat dengan mudah diteruskan ke dalam proses kompilasi berikutnya.

Lampiran

Link Github: https://github.com/raulinns/ISL-Tubes-IF2224-2026

Link Draw.io: https://drive.google.com/file/d/1r4e8K2wB8yXlr1yGBxgxNu1YmxVNyF0Z/view

Pembagian Tugas:

NIM	Nama	Bagian	Persentase Keseluruhan
13524044	Narendra Dharma Wistara M	Infrastructure DFA, literal, identifier, arithmetic	25%
13524060	Steven Tan	Keywords kontrol, komentar, error handling	25%
13524090	Nashiruddin Akram	Tanda baca, simbol struktur, logical operator	25%
13524100	Arghawisesa Dwinanda Arham	Declaration keywords, relational operator	25%

Referensi

[1] J. E. Hopcroft, R. Motwani, and J. D. Ullman, *Introduction to automata theory, languages, and computation*, 3. ed. Bosten San Francisco Munich: Pearson, 2007.

[2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Eds., *Compilers: principles, techniques, & tools*, 2. ed., Pearson internat. ed. Boston Munich: Pearson Addison-Wesley, 2007

[X] Laboratorium Ilmu Rekayasa dan Komputasi, "Spesifikasi Milestone 1 - Tubes IF2224 TBFO," Institut Teknologi Bandung, Bandung, 2026. [Online]. Available: https://docs.google.com/document/d/16dKSX_N3_pLFDT-C9qeZBtd-xERblDOmqh3EuuMvhrE/edit?usp=sharing

.